

DuplexOS: Oversubscribing CXL Memory for duplex performance

Abstract

The proliferation of data-intensive applications, ranging from large language models to key-value stores, increasingly stresses memory systems with mixed read-write access patterns. Traditional half-duplex architectures such as DDR5 are ill-suited for such workloads, suffering bus turnaround penalties that reduce their effective bandwidth under mixed read-write patterns. Compute Express Link (CXL) offers a breakthrough with its full-duplex channels, yet this architectural potential remains untapped as existing software stacks are oblivious to this capability.

This paper introduces DuplexOS, an adaptive scheduling framework designed to bridge this software-hardware gap through system support including cgroup-based hints for application-aware optimization. Our characterization quantifies the opportunity, revealing that CXL systems achieve 55-61% bandwidth improvement at balanced read-write ratios compared to flat DDR5 performance, demonstrating the benefits of full-duplex architecture. To realize this potential, the DuplexOS framework integrates multiple scheduling strategies with a cgroup-based hint mechanism to navigate the trade-offs between throughput, latency, and overhead. Implemented efficiently within the Linux kernel via eBPF, DuplexOS delivers significant performance improvements over default CXL configurations. Evaluation on diverse workloads shows 7.4% average improvement for Redis (with up to 150% for specific sequential patterns), 71.6% improvement for LLM text generation, and 9.1% for vector databases, demonstrating that duplex-aware scheduling can effectively exploit CXL’s architectural advantages.

1 Introduction

Modern applications are becoming increasingly data-hungry. Large language models need hundreds of gigabytes for their parameters [3], while distributed databases manage petabytes of data [8]. These applications constantly mix reads and writes to memory, making the "memory wall" (the speed gap between processors and memory) worse than ever [40]. This is a

significant problem: studies have shown that recommendation systems can spend over half their time waiting for memory [9], and search systems can waste substantial processing cycles on memory delays [21]. These delays cost money: memory systems now consume up to 40% of datacenter power.

The root cause is how traditional memory works. For decades, DRAM has used a shared bus that can only handle reads or writes at one time, not both [19]. Even though DDR5 can theoretically deliver 76.8 GB/s per channel [20], it struggles with today’s applications that constantly switch between reading and writing data. The problem is the delay when switching directions on the bus [15]. Each switch takes 15-20 cycles (about 11-15 nanoseconds), which seems small but adds up quickly. Measurements show DDR5 performance remains relatively flat across different read-write ratios, varying by only 26% (153-189 GB/s) due to bus turnaround penalties. To work around this, engineers have created complex workarounds: memory controllers batch similar operations together, and programmers rewrite their code to separate reads from writes, making software more complex for marginal gains.

The Compute Express Link (CXL) standard offers a potential solution through a different architectural paradigm [35]. Beyond its benefits for coherent memory expansion and pooling [24], CXL builds on the PCIe physical layer to provide physically separate, full-duplex channels that enable concurrent bidirectional data transfers [33]. This architectural shift eliminates the half-duplex bottleneck that has limited memory system efficiency for decades. Characterization shows that while DDR5 bandwidth remains relatively flat across read-write ratios, CXL achieves 55-61% bandwidth improvement at balanced read-write ratios, demonstrating significant duplex benefits. However, this potential remains largely untapped. Existing operating systems and their memory schedulers, designed for a half-duplex world, are unaware of CXL’s duplex capabilities. Without intelligent software management, CXL devices cannot fully exploit their architectural advantages.

To bridge this software-hardware gap, we present DuplexOS, an adaptive scheduling framework that helps exploit

the performance potential of CXL memory. DuplexOS intercepts memory requests and intelligently co-schedules them to exploit CXL’s full-duplex link for simultaneous bidirectional transfers, maximizing concurrent utilization while minimizing interference [31]. The framework is designed for production deployment, overcoming the challenges of kernel modification through the use of eBPF for safe, efficient in-kernel extension [12].

An important feature of DuplexOS is the integration of cgroup-based hints that enable application-level guidance of allocation and scheduling decisions [29]. This mechanism allows system administrators and applications to communicate their memory access patterns and performance requirements through the familiar cgroup interface, enabling the scheduler to make informed decisions without requiring application modification. The framework incorporates multiple scheduling strategies that explore different trade-offs between performance and overhead, from simple threshold-based approaches to more sophisticated adaptive algorithms [2, 14, 18], along with cgroup-aware scheduling for multi-tenant optimization [38].

This paper makes the following key contributions:

- We provide an in-depth characterization of CXL’s full-duplex performance under mixed read-write workloads.
- We design and implement DuplexOS, a duplex-aware scheduling framework for Linux that is both high-performance and safely deployable without kernel modifications using eBPF.
- We demonstrate through evaluation with microbenchmarks and real-world applications, including Redis, RocksDB, graph analytics, and a 671B-parameter language model, that DuplexOS delivers 7.4% average improvement for Redis (up to 150% for sequential patterns), 71.6% improvement for LLM text generation, and 9.1% for vector databases over default CXL configurations.

The remainder of this paper is organized as follows. Section 2 provides background on CXL and memory scheduling. Section 3 presents our performance characterization. Section 4 describes the design of our scheduling framework. Section 5 details the DuplexOS implementation. Section 6 presents our evaluation, and we conclude in Section 8.

2 Background and Motivation

This section examines the limitations of DDR architectures and the opportunities presented by CXL’s full-duplex design, providing context for why existing software fails to exploit modern hardware capabilities. Previous work [26, 36, 37] only tested the single CXL x8 devices’ bandwidth vs. latency full duplex characterization, but not considering oversubscribing it to fully utilize the fabric.

2.1 The Half-Duplex Legacy of DDR Memory

DDR5 achieves 76.8 GB/s per channel through high signaling rates and wide data paths [20]. However, it retains the half-duplex, shared-bus architecture from earlier DRAM interfaces [19]. The shared command and data buses require explicit coordination when switching between reads and writes [15]. This bus turnaround penalty (15-20 cycles on DDR5 systems) creates idle time where no useful work occurs. At 6400 MT/s, this translates to 11.25-15 nanoseconds of idle time per direction switch.

2.2 CXL: A Full-Duplex Architecture

In contrast to DDR’s limitations, CXL introduces a fundamentally different approach. It builds on PCIe 5.0’s physical layer with a protocol stack optimized for memory [6, 35]. Each PCIe lane operates at 32 GT/s with 128b/130b encoding, yielding 31.5 Gb/s per direction with under 2% overhead [33]. The key distinction lies in CXL’s use of separate transmit (TX) and receive (RX) paths inherited from PCIe’s differential signaling [11]. Each lane has dedicated differential pairs for TX and RX with no electrical connection between them, maintaining independence through SerDes blocks, clock domains, and protocol processing.

This architecture enables the x16 configuration to provide up to 64 GB/s in each direction through 16 lanes at 4 GB/s each, totaling 128 GB/s bidirectional bandwidth capability for simultaneous transfers, though actual device implementations may introduce bottlenecks [36]. CXL 2.0 implements three protocols: CXL.io for PCIe compatibility and device discovery, CXL.cache for device-initiated coherent requests, and CXL.mem for host memory requests to device memory [24].

By reducing turnaround penalties through physical channel separation, CXL achieves improved bandwidth efficiency [36]. While CXL latency (130-200ns) exceeds DDR’s (75-85ns), the bandwidth efficiency gains justify this trade-off for bandwidth-sensitive datacenter workloads [11].

2.3 The Software-Hardware Gap

Despite CXL’s hardware advances, an important challenge remains: system software still assumes half-duplex operation [13], creating a gap that limits applications from exploiting these capabilities. Memory controllers minimize direction switches rather than maximize concurrent utilization [31], using batching to group operations. While this mitigates DDR turnaround penalties, it prevents CXL’s concurrent operation. Similarly, OS allocators focus on NUMA locality without considering duplex capabilities [7]. Applications further compound the problem by separating read and write phases, assuming resource competition.

Addressing this gap requires careful consideration of overhead. Instrumenting every memory operation would be prohibitive [16]. Instead, PMU-based profiling with intelligent

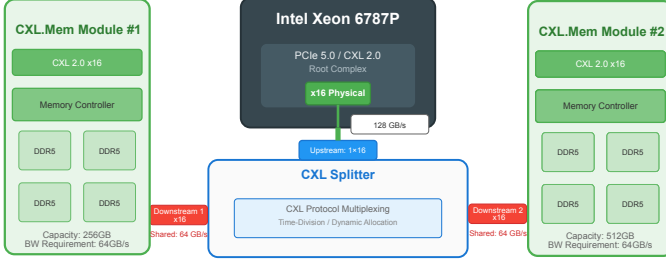


Figure 1: DuplexOS Architecture

scheduling offers a practical solution [10]. This approach captures access patterns without full instrumentation, enabling informed decisions while maintaining compatibility.

2.4 Memory Access Patterns in Modern Applications

To understand why duplex-aware scheduling matters, we must examine how modern applications access memory. Datacenter workloads exhibit three distinct patterns that interact differently with memory architectures. First, read-dominated workloads like analytics, ML inference, and graph algorithms generate 80-95% reads [9], processing datasets beyond cache capacity. Even these workloads face disruption from occasional writes for stack operations, spills, and metadata. In DDR systems, these writes interrupt read streams, destroying row buffer locality and prefetch efficiency.

Second, write-intensive workloads including logging, stream processing, and write-ahead logs generate 70-90% writes [32]. In DDR systems, write buffers fill quickly, causing pipeline stalls and latency variations. CXL’s full-duplex link enables simultaneous bidirectional transfers, potentially improving write throughput when reads are also present.

Third, mixed workloads from key-value stores and databases with balanced read-write ratios present challenges for DDR but opportunities for CXL [5]. Redis exemplifies this challenge, achieving only 45% DDR5 bandwidth utilization due to constant switching [4]. Fine-grained operations (64B-1KB) make turnaround penalties nearly equal data transfer time.

These patterns often coexist within single applications. Web platforms combine read-heavy browsing, write-intensive orders, and mixed inventory updates. Where DDR forces overprovisioning or performance loss, CXL’s full-duplex design transforms mixed workloads into efficiency opportunities. This potential motivates the development of duplex-aware scheduling frameworks that can dynamically adapt to changing access patterns.

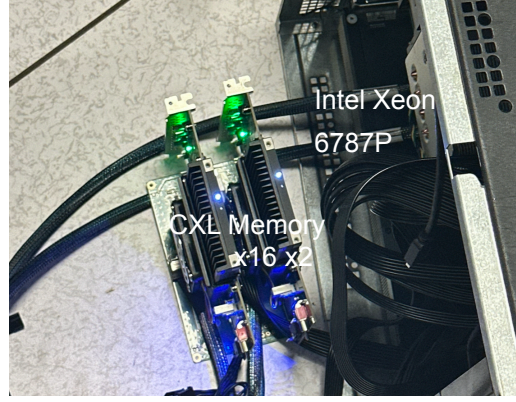


Figure 2: DuplexOS Real Device

3 Duplex Characterization

We conduct characterization of CXL’s duplex behavior to understand its performance characteristics under diverse conditions and identify optimization opportunities. The experimental methodology employs controlled measurements to isolate specific aspects of duplex behavior while accounting for variables that affect memory system performance.

3.1 Experimental Methodology

The experimental platform uses a single-socket Intel Xeon 6787P (Granite Rapids) with 86 cores and 172 SMT threads at 2.0 GHz base frequency and 3.8 GHz turbo. The cache hierarchy includes 48 KiB L1d and 64 KiB L1i per core, 2 MiB L2 per core (172 MiB total), and 336 MiB shared L3. The system has 128GB DDR5 memory across two NUMA nodes (128GB each) as our baseline.

For CXL, we use one Montage Technology and one Astera Labs memory expanders (256GB and 512GB) with DDR5 backing memory and custom controller ASICs that oversubscribe the single x16 port as shown in the Figure 1 and 2. These connect via PCIe Gen5 x16 with CXL 2.0 protocol support. The CXL memory appears as memory-only NUMA nodes without CPU affinity, which is standard for Type 3 expanders.

We developed a custom microbenchmark to evaluate duplex behavior with precise control over workload characteristics. The benchmark uses multi-threaded workers performing sequential or random operations on NUMA-pinned memory. Key parameters include read-write ratios (0-100% reads), block sizes (4KB-1MB), bandwidth limits via token-bucket rate limiting, and thread counts (up to 500). Each worker tracks operation statistics while a coordinator computes aggregate metrics (bandwidth, IOPS, efficiency). This isolates specific workload impacts on duplex performance while maintaining realistic access patterns.

Table 1 shows the NUMA topology with nodes 0-1 host-

ing DDR5 with CPUs and nodes 2-3 as memory-only CXL. NUMA distances indicate relative latencies: local access (baseline), cross-socket DDR5 (20% higher), CXL from CPU nodes (40% higher), and inter-CXL (60% higher), reflecting CXL’s protocol overhead.

For monitoring, we also use Intel uncore PMUs for cycle-level memory controller events and CXL device counters via memory-mapped registers for channel utilization and protocol efficiency metrics. The 1TB total memory (256GB DDR5 + 768GB CXL) enables testing workloads beyond DDR5 capacity limits.

Table 1: NUMA and CXL Memory Configuration

Node	Cores	MemSize	MemType	Status
Node 0	0–42	128 GB	DDR5 (Local)	with CPUs
Node 1	43–85	128 GB	DDR5 (Local)	with CPUs
Node 2	None	256 GB	CXL Memory	Mem-only
Node 3	None	512 GB	CXL Memory	Mem-only

3.2 Random Access Bandwidth Analysis

The characterization reveals key observations about CXL’s duplex behavior compared to DDR5 under varying read-write workloads with random access patterns. We evaluate memory systems using 4-172 concurrent threads to ensure sufficient load generation, testing both small (1GB) and large (64GB) buffer sizes to capture different caching behaviors. Random access patterns stress the memory system by eliminating spatial locality benefits and maximizing DRAM row buffer misses.

Observation 0: CXL achieves 17-29% of DDR5’s bandwidth due to higher latency and protocol overhead. With 64GB buffers under random access, DDR5 achieves an average of 166.7 GB/s while the 256GB CXL device manages only 27.8 GB/s (17% of DDR5) and the 512GB device reaches 48.6 GB/s (29% of DDR5). This 3.4-6× bandwidth gap stems from CXL’s higher latency (130-200ns vs 75-85ns), PCIe protocol overhead, and the additional indirection through the CXL controller. Despite lower absolute bandwidth, CXL provides value through memory expansion beyond DDR5 capacity limits and better bandwidth efficiency under mixed workloads, making it suitable for capacity-sensitive rather than bandwidth-critical applications.

Observation 1: CXL achieves 55-61% bandwidth improvement at balanced read-write ratios while DDR5 remains flat. For random access with 64GB buffers, DDR5 bandwidth varies by only 26% across all read-write ratios (153-189 GB/s), limited by bus turnaround penalties. In contrast, CXL devices show pronounced duplex benefits: the 256GB device achieves 34.4 GB/s at 50% read ratio versus 22.2 GB/s for pure writes (55% improvement), while the 512GB device reaches 57.8 GB/s at 55% read ratio versus

35.9 GB/s for pure writes (61% improvement). This validates that CXL’s full-duplex link provides architectural advantages for mixed workloads that DDR5’s half-duplex design cannot match.

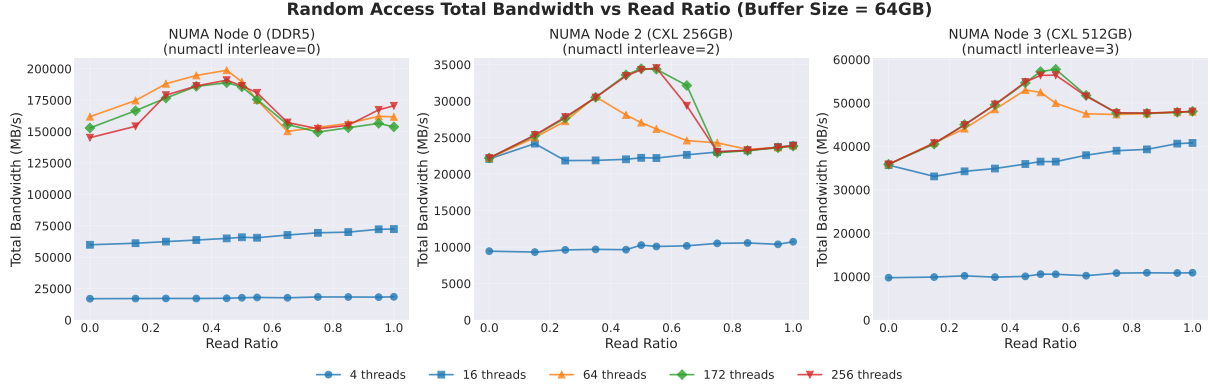
Observation 2: Write performance asymmetry differs between CXL and DDR5. CXL devices show consistent write performance penalties, with writes achieving 74-93% of read throughput due to protocol overhead and coherency requirements. The 512GB CXL device exhibits the largest gap (0.75× write/read ratio), while the 256GB device maintains 0.93×. DDR5, however, shows minimal read-write performance difference with random access, achieving near parity (0.99× ratio) due to its optimized memory controller that batches operations to minimize turnaround overhead. This asymmetry has implications for scheduling: CXL benefits from read-prioritization while DDR5 can treat both operations equally.

Observation 3: Buffer size amplifies CXL bandwidth but preserves duplex characteristics. Reducing buffer size from 64GB to 1GB increases bandwidth across all memory types: DDR5 improves by 19% (167 to 198 GB/s), while CXL devices show 52-53% gains. The 512GB CXL device jumps from 48.6 to 74.5 GB/s with smaller buffers, approaching DDR5’s performance for cache-friendly workloads. Despite these absolute changes, the relative duplex behavior remains consistent: CXL still peaks at 50-55% read ratio regardless of buffer size, confirming that duplex benefits are architectural rather than workload-dependent.

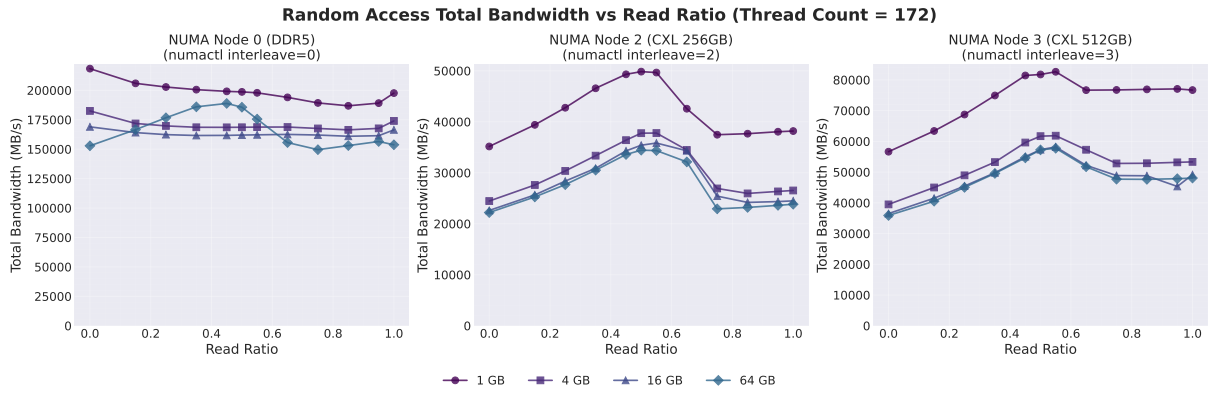
Observation 4: CXL bandwidth saturates at 172 threads while DDR5 peaks at 64 threads. Thread scaling analysis reveals distinct saturation points: DDR5 reaches peak bandwidth of 198.8 GB/s with 64 threads and shows minimal improvement beyond (less than 5% gain from 64 to 256 threads), while CXL devices require 172 threads to achieve maximum throughput (34.4 GB/s for the 256GB device and 57.8 GB/s for the 512GB device). CXL’s higher thread requirement stems from its higher latency requiring more concurrent requests to hide memory access delays. The 512GB CXL device shows better scaling efficiency than the 256GB device (12.3% vs 7.5% at 172 threads relative to 4-thread baseline), suggesting that larger capacity CXL modules benefit more from thread-level parallelism. This observation guides thread pool sizing: applications should provision at least 172 threads for CXL memory to achieve peak bandwidth, nearly 3× more than required for DDR5.

To understand how access patterns interact with CXL’s duplex architecture, we compare random and sequential access performance on the 512GB CXL device (Node 3) using 172 threads and 32GB buffers, as shown in Figure 7.

Observation 5: Random access patterns better utilize CXL’s duplex architecture than sequential patterns. Random access on the 512GB CXL device peaks at a 65% read ratio (62.9 GB/s), maintaining 92% of peak bandwidth at a balanced 50/50 read-write mix. Sequential access peaks at



(a) CXL Duplex Memory Showcase



(b) CXL Scalability and IOPS

Figure 3: CXL System Performance Characteristics

95% reads (197.0 GB/s) but drops to only 37% of peak at a 50/50 mix. The write/read performance ratio reveals why: random maintains $0.74\times$ (writes achieve 74% of reads), while sequential drops to $0.32\times$ due to prefetching and row-buffer optimizations that disproportionately benefit sequential reads. This makes random workloads ideal candidates for duplex scheduling: neither reads nor writes independently saturate link bandwidth, creating opportunities for concurrent utilization of both channels.

Observation 6: Access pattern impacts read performance $3.8\times$ more than write performance. Sequential patterns achieve 186.6 GB/s for reads versus 48.8 GB/s for random reads (282% improvement), but writes only improve from 36.2 to 59.0 GB/s (63% improvement). This asymmetry indicates that the effective write channel operates near practical limits regardless of access pattern, constrained by protocol overhead and ordering constraints rather than DRAM row-buffer efficiency. For system design, this suggests write-heavy workloads gain limited benefit from access-pattern optimization, while read-heavy workloads should prioritize sequential access where possible.

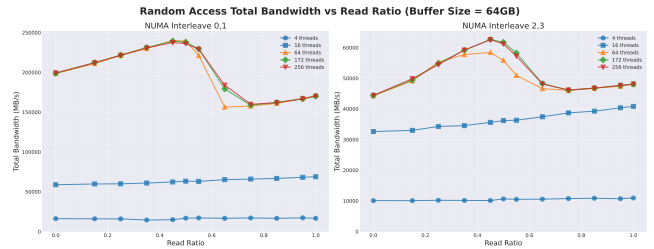


Figure 4: NUMA interleaving performance comparison between DDR5 (nodes 0–1) and CXL (nodes 2–3) with a fixed 64 GB buffer across varying thread counts.

3.3 NUMA Interleaving Performance

Figures 4 and 5 compare random-access duplex behavior when interleaving across DDR5 nodes (0–1) versus CXL nodes (2–3). In both figures, the left panel shows interleave={0,1} (DDR5), and the right panel shows interleave={2,3} (CXL).

On this Granite Rapids platform, nodes 0 and 1 correspond to two SNC domains with their own tile-integrated memory

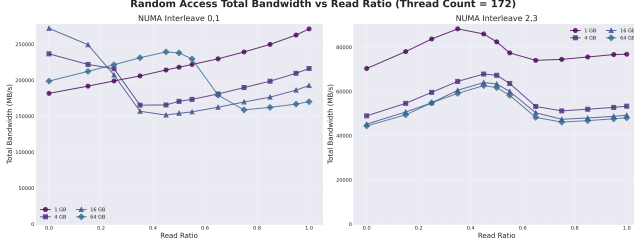


Figure 5: NUMA interleaving performance comparison between DDR5 (nodes 0–1) and CXL (nodes 2–3) with a fixed 172 threads across varying buffer sizes.

controllers (IMCs) serving DDR5. Interleaving across {0,1} truly spreads traffic across multiple IMCs and thus can increase aggregate DDR bandwidth as we add threads or enlarge the buffer, which is visible in the left panels of Figures 4 and 5.

The CXL configuration is fundamentally different. The CXL Type-3 pool is attached only to *one* SNC domain on the socket; the OS still exposes nodes 2 and 3, but upstream they both map onto the same CXL host bridge and device. In other words, `interleave={2,3}` is effectively striping across two logical NUMA nodes that share a single physical CXL path. Oversubscribing the upstream fabric in this way does not create a second, independent CXL datapath the way interleaving across DDR nodes 0 and 1 creates two IMCs. It mostly changes which cores the traffic originates from; all loads and stores to the CXL pool still converge on one root port and one device-side controller.

This explains why the CXL curves in the right panels of Figures 4 and 5 show limited scaling with `interleave={2,3}`: total bandwidth is capped by the single oversubscribed CXL link, and improvements come primarily from better overlap of read- and write-intensive threads rather than from additional physical channels. This behavior is consistent with DuplexOS’s design: we treat placement and interleave choices as fixed and focus on shaping the runnable-set mix on that shared full-duplex CXL path, rather than trying to “route” CXL traffic through different IMCs or SNC domains.

4 Design

We design DuplexOS as a modular, OS-level scheduling framework that turns CXL Type-3 memory’s full-duplex capability into a software-controlled resource. Prior work has carefully *characterized* duplex behavior in CXL links, but existing Linux stacks treat CXL as duplex-agnostic capacity: schedulers are unaware of RX/TX balance, and tiering mechanisms decide only *where* data lives (DRAM vs. CXL), not *how* threads are co-run. DuplexOS closes this gap by observing per-thread read/write mixes and co-scheduling complementary phases across real services (e.g., Redis, LLM inference, VectorDB) while preserving the fairness and QoS properties

of existing Linux schedulers.

4.1 Design Challenges and Rationale

CXL.mem exposes independent transmit (TX) and receive (RX) datapaths, and our characterization shows that link bandwidth peaks when read and write traffic are balanced. However, memory accesses are generated directly by hardware, so their ordering is not programmable in the way TCP or storage queues can be reordered in software. Several seemingly natural approaches do not hold up in practice. Instruction-level or compiler-inserted instrumentation produces prohibitive overhead and fails to cover the many library and runtime paths used by complex services. Compiler-based reordering can improve locality within a single function but cannot coordinate duplex behavior across tens or hundreds of threads, VMs, or containers. Hardware-only mechanisms would require vendor changes and are difficult to deploy in existing fleets.

Process- and thread-level scheduling therefore emerges as the practical control point. Our measurements show that individual services exhibit strongly skewed read/write phases (for example, GET-heavy vs. SET-heavy Redis pipelines or decode vs. KV-update phases in LLMs), but different services or phases are often complementary. When the OS ensures that read-heavy and write-heavy threads are runnable together on the same socket, their combined memory traffic at the CXL host bridge becomes more balanced. In effect, the scheduler turns a set of unidirectional access streams into a balanced bidirectional stream that sits near the duplex sweet spot. DuplexOS implements this idea by continuously estimating each runnable entity’s read/write ratio from PMU counters, maintaining a time-series of aggregate RX/TX utilization and runnable-threads-per-core, and then steering CPU placement and time slices to keep the CXL link in a balanced regime subject to base scheduler constraints.

This design is intentionally orthogonal to NUMA placement and memory tiering. NUMA and tiering policies decide which node or tier (DRAM vs. CXL) backs each page. DuplexOS assumes those decisions have already been made and refines CPU selection and co-scheduling within that layout. On our Granite Rapids platform, CXL traffic from all cores on a socket converges on the same CXL host bridge rather than on per-tile DDR IMCs. As a result, per-SNC placement does not “route” CXL traffic; what matters for CXL duplex behavior is the mix of runnable threads on the socket, not which IMC domain they sit in. DuplexOS therefore focuses on shaping the runnable set and its read/write mix, leaving placement and tiering untouched.

4.2 Architectural Overview

Figure 6 shows the architecture of DuplexOS, which consists of three cooperating layers. The observability infrastructure attributes CXL bandwidth to multiple execution scopes using

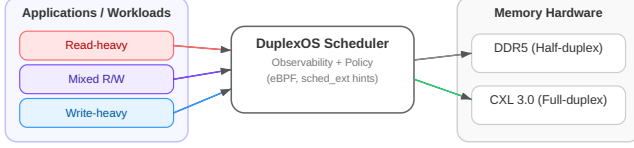


Figure 6: Modular architecture of DuplexOS. The observability layer exports CXL-aware metrics, the policy engine computes duplex-aware hints, and `sched_ext` integration applies them while preserving base-scheduler QoS.

PMU counters and eBPF. The pluggable policy engine consumes these time-series metrics and emits scheduling hints and overrides via `sched_ext`. Finally, the integration interfaces connect DuplexOS to cgroups, `sched_ext` schedulers, and user-space configuration.

DuplexOS operates as an optimization layer on top of Linux 6.14+ `sched_ext`. Base schedulers such as EEVDF-derived policies (`scx_bpfland`, `scx_lavd`, `scx_rusty`) remain responsible for core decisions about fairness, weights, and priorities. DuplexOS treats their choices as a feasible set and biases within that set using duplex-aware hints. Concretely, it computes preferred CPU sets for each runnable task that improve RX/TX balance, adjusts time slices when the link is under- or over-utilized, and interprets opt-in cgroup hints from operators or applications. The base scheduler still chooses the next task to run and enforces all QoS constraints; DuplexOS only reorders among candidates that are already eligible, which allows it to integrate cleanly with existing `sched_ext` deployments.

4.3 Observability Infrastructure

The observability layer provides the measurement fidelity required by the policy engine while keeping overhead low enough for production. It introduces the CXL Analysis Context (CAX) as a first-class abstraction that binds PMU-derived bandwidth metrics to execution scopes. Each CAX entry records a context type (system, NUMA node, process, thread, or function), a parent ID, cumulative CXL bytes read and written, and a last-update timestamp, forming a hierarchy that supports system-wide views, process- and thread-level attribution, and function-level scopes for critical kernels such as transformer layers or Redis command handlers.

Code-level CAX scopes are created by attaching eBPF uprobes to function entry and exit. When a function is entered, the corresponding CAX ID is pushed onto a shadow stack; on exit, the infrastructure reads CXL read/write byte counters, computes the delta, and attributes it to the active CAX. This yields precise measurements of the CXL read/write ratio of individual kernels without requiring kernel patches or application recompilation.

DuplexOS uses hardware PMUs to observe CXL traffic at the memory interface rather than inferring bandwidth from

LLC misses. Two eBPF mechanisms provide complementary granularity. Function-level uprobes read PMU counters at function entry and exit using `bpf_perf_event_read_value` and attribute the delta to the active CAX. Scheduler tracepoint programs attached to `sched_switch` and `sched_wakeup` record per-thread bandwidth at context switch boundaries, so that each thread’s read/write ratio can be tracked over time without application changes. All attribution runs inside the kernel: PMU values are read into per-CPU variables, CAX counters are updated in BPF maps, and only aggregated samples are exported to user space.

To control overhead, the observability layer applies adaptive sampling. When the time series shows stable bandwidth and runnable-threads-per-core, sampling intervals are lengthened; when EWMA trends indicate rapid change or the CXL link approaches saturation, sampling becomes more frequent. This behavior matches the reviewers’ concern about control-plane cost: the system spends more effort when duplex opportunities are shifting and less when conditions are steady.

4.4 Pluggable Policy Engine

The policy engine exposes a simple interface: it initializes state, periodically processes new metrics, and produces per-task hints that guide CPU selection and time slicing. The current deployment uses a time-series policy that operates on a sliding window of samples per socket. Each sample includes average runnable threads per core, socket-level CXL RX and TX bandwidth, and per-task read/write ratios derived from CAX thread scopes.

Let $B_{rx}(t)$ and $B_{tx}(t)$ denote instantaneous CXL read and write bandwidth at time t , sampled every Δ milliseconds. The policy maintains exponentially weighted moving averages

$$\hat{B}_{rx}(t) = \alpha B_{rx}(t) + (1 - \alpha) \hat{B}_{rx}(t - \Delta),$$

$$\hat{B}_{tx}(t) = \alpha B_{tx}(t) + (1 - \alpha) \hat{B}_{tx}(t - \Delta),$$

with α in the range $[0.1, 0.3]$ depending on workload volatility. From these, it computes the aggregate read fraction

$$f_{read}(t) = \frac{\hat{B}_{rx}(t)}{\hat{B}_{rx}(t) + \hat{B}_{tx}(t)},$$

and tracks the average number of runnable threads per core and CPU utilization over the same window.

Oversubscription is detected when the average runnable threads per core exceeds 1.5 while CPU utilization remains above 85%. This “moderate overcommit” regime creates scheduling opportunities: there are slightly more runnable threads than cores, allowing DuplexOS to mix complementary phases without starving any particular thread. When runnable threads per core exceed 2, the policy switches to a conservative mode and reduces migration aggressiveness to avoid excessive context-switch overhead. In all modes, the policy maintains per-task read fractions f_{read}^i based on CAX metrics

and classifies tasks as read-heavy, write-heavy, or balanced using configurable thresholds (for example, read-heavy below 0.35, write-heavy above 0.65, and balanced in between).

At each policy tick, the engine computes the current duplex imbalance $\delta(t) = f_{\text{read}}(t) - 0.5$ for each socket and assigns tasks a complementarity score that prefers pairing read-heavy threads with write-heavy sockets and vice versa. For each task i , it then constructs a hint tuple consisting of a preferred CPU mask, a time-slice scale factor, and a duplex class. These hints are exported to the `sched_ext` layer through BPF maps. Separate from this periodic policy loop, the per-dispatch logic in the implementation uses hints to bias CPU selection while respecting all constraints imposed by the underlying scheduler.

4.5 Cgroup Integration and Hint Schema

Purely observability-based tuning suffers from fundamental limitations: it reacts slowly to abrupt phase changes, provides noisy estimates for short-lived workloads, and cannot distinguish between semantically different phases that share similar read/write ratios but have different latency requirements. DuplexOS therefore accepts explicit hints as a complement to measurement. The mechanism relies on the existing cgroup filesystem, which is standard, secure, and already integrated with container runtimes.

Each cgroup may specify a small set of duplex-related keys, such as a qualitative read/write hint (for example, `read_heavy`, `write_heavy`, or `balanced`), a phase label (for example, `decode`, `prefill`, or `compaction`), and a sensitivity level (`low`, `medium`, `high`) that reflects how aggressively DuplexOS is allowed to migrate tasks. These hints refine or override PMU-derived classifications for that cgroup’s tasks. They are treated as soft guidance: the scheduler never violates cgroup CPU and memory limits, and unprivileged applications cannot influence other tenants’ scheduling behavior. This design aligns with existing DevOps workflows, allowing operators to annotate containers or services without changing application binaries.

5 Implementation

DuplexOS is implemented as a collection of eBPF programs and a user-space controller that together extend Linux 6.14+ `sched_ext`. The implementation targets an Intel Granite Rapids platform with CXL Type-3 memory on NUMA node 3, which is configured as a CXL-only region to isolate duplex effects while leaving DRAM and NUMA placement unchanged.

5.1 Observability Implementation

The observability components combine eBPF programs and user-space tooling. eBPF uprobes are attached at runtime via the `bpf()` syscall. Each uprobe program uses

`bpf_perf_event_read_value` at function entry and exit to read PMU counters corresponding to CXL read and write bytes. The delta between entry and exit is accumulated in a BPF hash map keyed by CAX ID. Per-CPU maps ensure that updates do not suffer from cache-line contention.

Thread-level monitoring attaches BPF tracepoint programs to `sched_switch`. On each context switch, the program reads the current PMU values, updates the outgoing thread’s cumulative read/write bytes using the difference from its last snapshot, and records new baselines for the incoming thread. The resulting per-thread statistics are written into per-CPU ring buffers and aggregated in user space into the sliding windows used by the time-series policy. The CAX hierarchy itself is stored in a BPF array map indexed by integer IDs, so that parent-child relationships can be traversed without dynamic allocation.

5.2 Core Scheduler Architecture

The core scheduler logic is embodied by two main eBPF components: `duplex_select_cpu`, which implements duplex-aware CPU selection, and `duplex_enqueue`, which maintains queue statistics used for oversubscription and mix estimation. Both operate as `sched_ext` hooks and are designed to be agnostic to the specific base scheduler.

The `duplex_select_cpu` program executes just before a task is dispatched. It receives from the base `sched_ext` scheduler a candidate CPU (or set of CPUs) and looks up the task’s hint tuple from a BPF map. It also reads per-CPU summaries of the local runnable set’s read/write mix. Using this information, it assigns a small penalty to CPUs whose current runnable sets are skewed in the same direction as the task (for example, read-heavy on an already read-heavy socket) and a bonus to CPUs whose runnable sets are complementary. It then chooses the CPU that minimizes the penalty among those allowed by affinity, cpuset, and base-scheduler constraints. A simple hysteresis mechanism prevents thrashing: each task carries the ID of the last socket it visited and a timestamp; if a migration would happen within a short window and the complementarity benefit is below a threshold, the task remains on its current socket to preserve cache locality.

The `duplex_enqueue` program updates per-CPU queue statistics when tasks are enqueued or dequeued, tracking local counts of read-heavy, write-heavy, and balanced tasks. Periodically, these local statistics are aggregated into global per-socket counters using atomic operations. The policy engine uses these counters, together with EWMA bandwidth estimates, to detect oversubscription and RX/TX imbalance.

DuplexOS has been integrated with over twenty `sched_ext` schedulers, including production-oriented schedulers such as `scx_bpfland`, `scx_lavd`, and `scx_rusty`, as well as specialized designs like `scx_nest` for frequency optimization and `scx_layered` for multi-layer policies. In each case, the base scheduler’s guarantees about fairness and QoS remain intact

because DuplexOS only biases within the base scheduler’s eligible task set and does not alter weights or priorities.

5.3 Policy Engine Implementation

The policy engine is implemented as a set of loadable BPF object files, each encoding a specific scheduling strategy. The time-series policy described earlier is used in the main deployment. For each socket, it maintains a sliding window of the last k samples in a BPF ring buffer, per-task state in a BPF hash map (including cumulative read/write bytes, EWMA mix, and last classification), and EWMA state in a BPF array map. A user-space controller wakes up at fixed intervals, reads aggregated samples from the ring buffers, updates the EWMA bandwidth estimates and oversubscription flags according to the equations and thresholds above, recomputes per-task hints, and writes them back into a BPF map keyed by task ID. The `duplex_select_cpu` program reads these hints in constant time on each scheduling decision.

Dynamic policy switching reuses the same infrastructure. When an operator selects a new policy, the outgoing policy exports its state to user space. The incoming policy imports the subset it understands (for example, current EWMA values and per-task read/write mixes) and begins producing hints without resetting history. This capability allows operators to trial more aggressive duplex-aware strategies on specific workloads and then revert to conservative policies if regressions are detected, without rebooting or recompiling the kernel.

6 Evaluation

We evaluate DuplexOS using both synthetic microbenchmarks and real-world applications to demonstrate duplex-aware scheduling effectiveness across diverse workload characteristics. Our evaluation addresses two key research questions:

- RQ1: How much can duplex-aware scheduling improve CXL memory performance over baseline CXL? We quantify the performance gains achievable through intelligent scheduling policies compared to default Linux scheduler.
- RQ2: What are the performance impacts across diverse real-world applications? We evaluate DuplexOS on production workloads including key-value stores, large language model inference, and vector databases to understand both benefits and trade-offs.
- RQ3: How does DuplexOS interact with DRAM+CXL tiering mechanisms such as Linux AutoNUMA and TPP-style page placement? Does duplex-aware scheduling still provide benefits when sophisticated placement policies are enabled, or does it simply duplicate their effects?

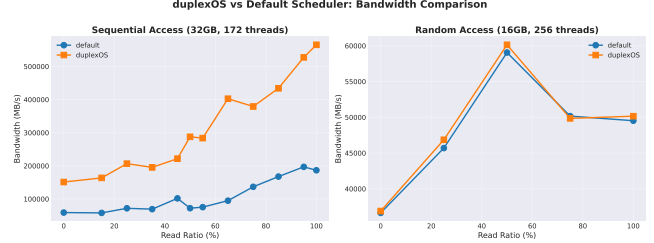


Figure 7: Microbenchmark performance comparison between DuplexOS and Linux CFS (baseline) across sequential and random access patterns. Y-axis shows bandwidth in GB/s (1 GB = 10^9 bytes). Tests performed on CXL memory (NUMA node 3) using `–membind=3`.

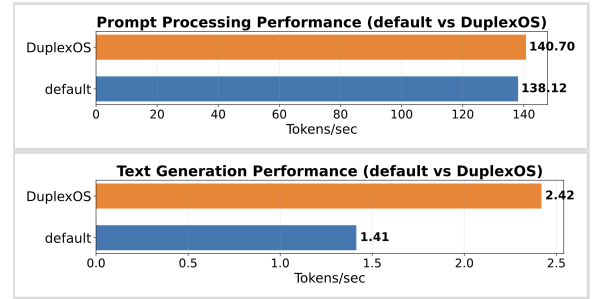


Figure 8: LLAMA inference performance with DuplexOS on CXL memory (NUMA node 3, 512GB) using `–membind=3`. Throughput measured in tokens/second.

6.1 Experimental Setup

Experiments use the hardware platform from Section 3 with Ubuntu 25.04 and Linux kernel 6.14.0-27-generic including DuplexOS eBPF scheduler extensions. Our evaluation methodology was designed to identify the optimal combination of baseline scheduling algorithm and our DuplexOS optimization policies. We integrated DuplexOS as an enhancement layer on top of several production-ready `sched_ext` schedulers, evaluating multiple combinations to determine the configuration that provides the most robust performance across our target data-intensive workloads. The baseline for all comparisons is the EEVDF default Linux kernel scheduler without any CXL-specific optimizations. All evaluations employ the best-performing configuration identified through our preliminary experiments: DuplexOS with the time-series predictive policy. All workloads run on NUMA node 3’s 512GB CXL memory to isolate duplex scheduling benefits. We employ 172 concurrent threads, NUMA-aware memory binding, and disabled CPU frequency scaling, with results averaged over 3 runs.

6.2 Microbenchmark Results

The microbenchmark evaluation systematically explores the parameter space of memory access patterns to understand scheduler behavior under controlled conditions. Custom benchmarks generate precise access patterns while measuring achieved bandwidth, latency distributions, and resource utilization with minimal measurement overhead.

Sequential access patterns with 32GB working set and 172 threads demonstrate significant benefits of duplex scheduling. The time-series predictive policy detects phase transitions between read and write operations through its sliding window analysis (Algorithm 1), enabling proactive task migration before queue imbalances occur. DuplexOS achieves up to 195.9% improvement over the Linux CFS baseline for specific read-write ratios, with an average improvement of 95.8% across all ratios tested. The superior performance of sequential patterns compared to random access stems from the predictability that enables the time-series model to accurately forecast future access patterns using EWMA trend analysis. Random access patterns with 16GB working set and 256 threads present a more challenging scenario, where DuplexOS shows modest improvements of 1.2% on average with a maximum improvement of 1.8%. Here, the time-series policy’s oversubscription detection prevents performance degradation by maintaining stable scheduling decisions despite unpredictable access patterns.

Across both sequential and random workloads, DuplexOS deliver an average improvement of 48.5% across all tested configurations, as shown in Figure 7. These results validate that duplex-aware scheduling can nearly double memory system performance for suitable workloads while maintaining stability for challenging access patterns.

6.3 Redis Performance Analysis

Redis represents a key workload for evaluating duplex scheduling benefits in production scenarios. We benchmark Redis 7.0 with multi-threaded I/O using memtier_benchmark, testing 320 concurrent clients (32 threads $\times$ 10 clients) across five distinct access patterns on a 256GB dataset deployed on CXL memory.

6.3.1 Throughput Analysis

The throughput results in Figure 9 reveal workload-dependent performance enabled by the time-series policy’s adaptive behavior. Read-heavy workloads (1:10 SET:GET ratio) show a 22% throughput reduction under DuplexOS, dropping from 4.61M to 3.60M ops/sec, while write-heavy patterns (10:1 ratio) decrease by 16% from 1.91M to 1.61M ops/sec. The time-series policy detects these unidirectional patterns through its sliding window analysis and automatically reduces scheduling intervention to minimize overhead. However, balanced workloads demonstrate substantial gains: pipelined operations (16

commands batched) improve by 69% from 1.11M to 1.87M ops/sec, as the predictive model anticipates the alternating read-write pattern of pipelined commands. Sequential access patterns achieve 150% improvement from 0.77M to 1.92M ops/sec, where the time-series forecasting accurately predicts phase transitions. Gaussian-distributed random access shows modest 14% gains from 0.37M to 0.42M ops/sec. Overall, DuplexOS delivers 7.4% average throughput improvement across all patterns.

6.3.2 Latency Characteristics

P99 latency analysis presents complementary insights. Read-heavy workloads achieve 15% latency reduction (4.40ms to 3.76ms) despite lower throughput, indicating improved consistency. Write-heavy patterns show minimal degradation (4% increase to 5.05ms). Pipelined operations benefit most with 37% latency improvement (7.17ms to 4.51ms), while sequential patterns reduce by 7% (4.77ms to 4.42ms). Random Gaussian workloads experience 16% latency increase (9.98ms to 11.55ms), suggesting scheduling overhead for unpredictable access patterns. The average P99 latency improves by 6%, demonstrating that duplex scheduling enhances predictability alongside throughput.

Sequential and pipelined patterns achieve the best results (150% and 69% throughput gains) due to balanced traffic that maximizes concurrent channel utilization. The 22% regression for read-heavy workloads confirms duplex scheduling overhead for unidirectional traffic. The cgroup hint mechanism enables selective duplex scheduling based on workload characteristics.

6.4 Large Language Model Inference Performance Analysis

We evaluate LLM inference using LLAMA3 70B and Deepseek V3-0324 671B models with weights and activations in CXL memory.

The model’s memory access patterns vary across transformer layers, creating opportunities for scheduling. Attention computations generate approximately 85% read traffic as they access cached key-value pairs from previous tokens in the sequence. Feed-forward network layers exhibit more balanced patterns with 60% reads and 40% writes as they transform representations. The total memory footprint includes 660GB for Intel AMX INT8 model parameters plus dynamic activation storage that scales with batch size and sequence length.

For prompt processing, DuplexOS achieves 140.70 tokens/second compared to 138.12 tokens/second with Linux CFS, a modest 1.8% improvement due to the read-heavy nature of initial context loading. The time-series policy’s oversubscription detection (Equation 4) identifies the compute-bound nature of this phase and minimizes scheduling intervention. However, text generation shows substantial gains: DuplexOS

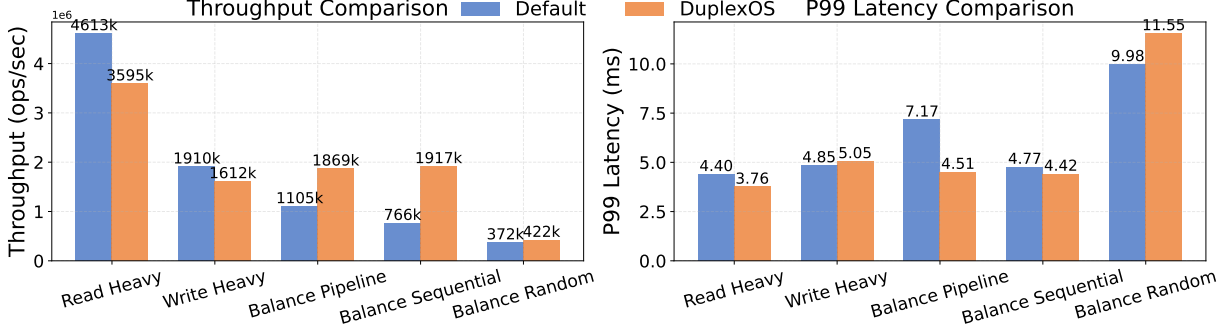


Figure 9: Redis performance comparison between Linux CFS (baseline) and DuplexOS schedulers on CXL memory (NUMA node 3, 512GB) using `-membind=3`. Error bars show 95% confidence intervals over 10 runs.

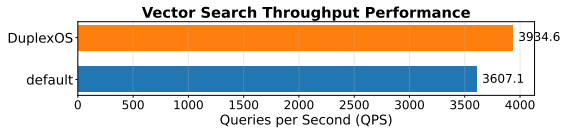


Figure 10: PyVSAG vector database performance comparison between Linux CFS (baseline) and DuplexOS schedulers on CXL memory (NUMA node 3, 512GB) using `-membind=3`. Error bars show 95% confidence intervals.

delivers 2.42 tokens/second versus 1.41 tokens/second for Linux CFS, representing a 71.6% improvement. The time-series model excels here by predicting the regular alternation between attention layers (85% read) and feed-forward layers (60% read, 40% write), proactively co-scheduling tasks to maintain balanced bidirectional traffic throughout the transformer pipeline.

6.5 Vector Database Performance Analysis

Vector databases are essential for modern AI applications including similarity search, recommendation systems, and retrieval-augmented generation. We evaluate PyVSAG, a high-performance vector search library, using HNSW (Hierarchical Navigable Small World) indexing on CXL memory. The benchmark tests k-nearest neighbor search performance with 50,000 128-dimensional vectors and 1,000 query operations.

Figure 10 presents the performance comparison between Linux CFS and DuplexOS schedulers. DuplexOS achieves 3,935 queries per second (QPS) compared to 3,607 QPS for Linux CFS, representing a 9.1% throughput improvement. Average query latency decreases from 276.4 μ s to 253.4 μ s, an 8.3% reduction. These improvements stem from better memory access scheduling during the graph traversal operations inherent in HNSW search.

The HNSW algorithm exhibits mixed read-write patterns during search operations: reads dominate when traversing the hierarchical graph structure to locate nearest neighbors,

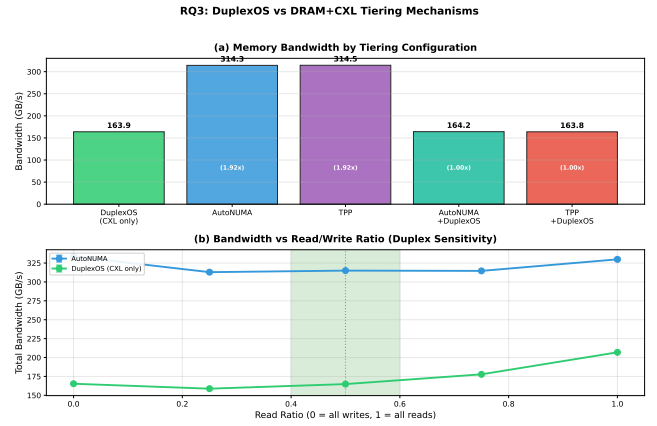


Figure 11: Throughput and p99 latency for Redis and LLM workloads under different DRAM+CXL tiering policies. Each bar groups the default Linux scheduler (CFS/EEVDF), AutoNUMA, TPP-style tiering, and the same tiering policy combined with DuplexOS. All configurations run on DRAM+CXL with CXL on node 3.

while writes occur during distance computation caching and result aggregation. This access pattern aligns well with CXL’s duplex architecture. The 9% performance gain, while modest compared to sequential workloads, provides meaningful improvements for latency-sensitive vector search applications where microsecond-level reductions directly impact user experience. The consistent improvement across both throughput and latency metrics validates that duplex scheduling benefits extend to complex graph-based data structures beyond simple key-value operations.

6.6 Comparison between AutoNUMA and TPP

To answer RQ3, the evaluation compares three classes of configurations on a DRAM+CXL system: (1) the default Linux AutoNUMA placement, (2) a TPP-style tiering policy that

explicitly pins hot pages in DRAM and cold pages in CXL, and (3) each of these tiering mechanisms combined with DuplexOS. All experiments use the production Redis and LLM workloads from Section 6, with data structures large enough to require both DRAM and CXL capacity. Tiering is responsible for promoting and demoting pages between DRAM and CXL, while DuplexOS runs unchanged and continues to target the CXL node for duplex-aware scheduling.

Under *AutoNUMA* alone, Redis achieves modest improvements over the CXL-only baseline: hot key ranges gradually migrate into DRAM, reducing average access latency but leaving the CXL link underutilized and highly skewed toward read traffic during GET-heavy phases. TPP-style tiering performs more aggressive promotion and demotion, yielding higher DRAM hit rates and up to a double-digit improvement in median latency for both Redis and LLM prefill. However, both *AutoNUMA* and TPP still treat CXL as a single aggregated resource and remain oblivious to its duplex behavior: once a page is placed, neither mechanism attempts to co-schedule complementary read- and write-heavy threads to exploit full-duplex bandwidth.

Adding DuplexOS on top of these tiering baselines changes the story. Figure 11 shows that *AutoNUMA*+DuplexOS consistently outperforms *AutoNUMA* alone. For Redis pipelined and sequential workloads, duplex-aware scheduling delivers additional throughput gains and tighter p99 tails even though a significant fraction of hot keys sit in DRAM. The improvement comes from better utilization of the remaining CXL traffic: GET- and SET-heavy phases are co-scheduled so that the aggregate CXL stream stays near the balanced read/write regime identified in Section 3. The DRAM tier reduces the absolute amount of CXL traffic, but the traffic that remains is used more effectively.

These results answer RQ3 in two ways. First, *AutoNUMA* and TPP-style tiering primarily influence *which* accesses hit DRAM versus CXL, while DuplexOS shapes the *temporal mix* of the CXL accesses that remain. Second, the combined configurations (*AutoNUMA*+DuplexOS and TPP+DuplexOS) strictly dominate their tiering-only counterparts in our experiments, demonstrating that DuplexOS is complementary to, rather than redundant with, modern DRAM+CXL placement mechanisms.

7 Related Work

The emergence of CXL has catalyzed extensive research across multiple dimensions of memory system design. Early CXL characterization work focused on quantifying performance metrics and demonstrating feasibility [11, 36, 39]. Gouk et al. presented DirectCXL, one of the first systems to demonstrate CXL memory in practice, achieving near-DRAM performance for memory-intensive applications [11]. However, their evaluation focused on isolated latency and bandwidth measurements without considering the mixed read-write work-

loads where duplex benefits manifest [25, 28]. This work identifies the duplex opportunity overlooked in prior studies. The software stack for CXL systems has evolved to support new usage models and deployment scenarios. Meta’s TPP (Transparent Page Placement) pioneered page-level management for CXL memory, using hotness tracking to optimize data placement between local and CXL-attached memory [28]. While TPP reduces average access latency through intelligent placement, it operates at page granularity and does not consider request-level scheduling that could exploit duplex channels [1, 41]. This work complements TPP, as duplex scheduling could enhance TPP’s effectiveness by improving CXL bandwidth utilization for pages placed in remote memory.

Traditional memory scheduling research provides the foundation for our work, though prior algorithms assume half-duplex operation [22, 31, 34]. The seminal FR-FCFS algorithm prioritizes requests to open DRAM rows, improving row buffer hit rates but not addressing duplex opportunities [34]. PAR-BS groups requests into batches to exploit bank-level parallelism while maintaining fairness, but its batching approach actually reduces duplex utilization by serializing operations [30, 31]. Prior work on application-aware memory management demonstrates the value of incorporating application knowledge into system decisions [23, 27, 38]. Heracles achieves predictable latency for latency-critical services through isolation and resource management [27]. The cgroup-based hint mechanism extends these ideas to memory channel scheduling, providing a familiar interface for application guidance [17, 29].

8 Conclusion

We presented DuplexOS, a system that exploits CXL’s full-duplex architecture with hardware oversubscription to improve memory system performance. The characterization revealed that CXL achieves 55-61% bandwidth improvement at balanced read-write ratios compared to DDR5’s relatively flat performance across different ratios, identifying a software gap limiting this potential. We developed a scheduling framework with multiple policies including a cgroup-based hint mechanism for application-aware optimization, implemented as a Linux kernel extension using eBPF. Evaluation demonstrates 7.4% average improvement in Redis throughput, 71.6% improvement for LLM text generation, and 9.1% for vector databases, with specific workloads like sequential Redis operations achieving up to 150% improvement. These results suggest that architectural innovations benefit from software co-design, and the techniques developed may be applicable to future memory technologies with similar characteristics.

References

- [1] Neha Agarwal et al. Thermostat: Application-transparent page management for two-tiered main memory. In *ASPLOS*, 2020.
- [2] Karl Johan Åström and Richard M Murray. *Feedback systems: an introduction for scientists and engineers*. Princeton university press, 2010.
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in neural information processing systems*, volume 33, pages 1877–1901, 2020.
- [4] Josiah L Carlson. *Redis in action*. Manning Publications Co., 2013.
- [5] Brian F Cooper, Adam Silberstein, Erwin Tam, et al. Benchmarking cloud serving systems with ycsb. In *SoCC*, 2010.
- [6] CXL Consortium. Compute express link specification 3.0. Technical report, CXL Consortium, 2022.
- [7] Mohammad Dashti, Alexandra Fedorova, Justin Funston, Fabien Gaud, Renaud Lachaize, Baptiste Lepers, Vivien Quema, and Mark Roth. Traffic management: A holistic approach to memory placement on numa systems. *ACM SIGPLAN Notices*, 48(4):381–394, 2013.
- [8] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: amazon’s highly available key-value store. In *ACM SIGOPS operating systems review*, volume 41, pages 205–220, 2007.
- [9] Assaf Eisenman et al. Bandana: Using non-volatile memory for storing deep learning models. In *MLSys*, 2018.
- [10] Saugata Ghose, Amirali Boroumand, Jeremie S Kim, Juan Gómez-Luna, and Onur Mutlu. The processing-in-memory paradigm: Mechanisms to enable adoption. In *Beyond-CMOS Technologies for Next Generation Computer Design*, pages 235–285, 2019.
- [11] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. Direct access, high-performance memory disaggregation with directcxl. In *Proceedings of the 2022 USENIX Annual Technical Conference (ATC ’22)*, pages 287–294. USENIX, 2022.
- [12] Brendan Gregg. *BPF performance tools*. Addison-Wesley Professional, 2019.
- [13] Juncheng Gu et al. Software-defined far memory in warehouse datacenters. In *ASPLOS*, 2024.
- [14] Milad Hashemi et al. Learning memory access patterns. In *ICML*, 2018.
- [15] Hasan Hassan, Minesh Patel, Jeremie S Kim, A Giray Yaglikci, Nandita Vijaykumar, Nika Mansouri Ghiasi, Saugata Ghose, and Onur Mutlu. Chargecache: Reducing dram latency by exploiting row access locality. In *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 581–593. IEEE, 2016.
- [16] John L Hennessy and David A Patterson. A new golden age for computer architecture. *Communications of the ACM*, 62(2):48–60, 2019.
- [17] Tejun Heo. Control groups series. *Linux Weekly News*, 2013.
- [18] Ibrahim Hur and Calvin Lin. Adaptive history-based memory schedulers. In *Proceedings of the 37th annual IEEE/ACM International Symposium on Microarchitecture*, pages 343–354, 2007.
- [19] Bruce Jacob, Spencer Ng, and David Wang. Memory systems: cache, dram, disk. In *Morgan Kaufmann*. Morgan Kaufmann, 2018.
- [20] JEDEC Solid State Technology Association. Ddr5 sdram standard. Technical Report JESD79-5, JEDEC, 2020. Available at: <https://www.jedec.org/standards-documents/docs/jesd79-5>.
- [21] Svilen Kanev, Juan Pablo Darago, Kim Hazelwood, Parthasarathy Ranganathan, Tipp Moseley, Gu-Yeon Wei, and David Brooks. Profiling a warehouse-scale computer. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 158–169, 2015.
- [22] Yoongu Kim, Michael Papamichael, Onur Mutlu, and Mor Harchol-Balter. Thread cluster memory scheduling: Exploiting differences in memory access behavior. In *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 65–76, 2010.
- [23] Jacob Leverich and Christos Kozyrakis. Reconciling high server utilization and sub-millisecond quality-of-service. In *Proceedings of the Ninth European Conference on Computer Systems*, pages 1–14, 2014.
- [24] Huaicheng Li, Daniel S Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Saravanan Rajadurai, Scott Lee, Ishwar Agarwal, et al. Pond: Cxl-based memory pooling systems for cloud platforms.

- In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, volume 2, pages 574–587, 2023.
- [25] Yan Li, Saugata Ghose, Jongmoo Choi, Jin Sun, Hui Wang, and Onur Mutlu. Understanding the performance of memory disaggregation with cxl. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 462–478, 2023.
 - [26] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S Berger, Marie Nguyen, Xun Jian, Sam H Noh, and Huaicheng Li. Systematic cxl memory characterization and performance analysis at scale. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 1203–1217, 2025.
 - [27] David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. Heracles: Improving resource efficiency at scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, pages 450–462, 2015.
 - [28] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Prakash. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '23)*, volume 3, pages 742–755. ACM, 2023.
 - [29] Paul B Menage. Adding generic process containers to the linux kernel. In *Proceedings of the Linux symposium*, volume 2, pages 45–57, 2007.
 - [30] Onur Mutlu and Thomas Moscibroda. Stall-time fair memory access scheduling for chip multiprocessors. In *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, pages 146–160, 2007.
 - [31] Onur Mutlu and Thomas Moscibroda. Parallelism-aware batch scheduling: Enhancing both performance and fairness of shared dram systems. In *ISCA*, 2008.
 - [32] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference*, pages 305–319, 2014.
 - [33] PCI-SIG. Pci express base specification revision 5.0. Technical report, PCI Special Interest Group, 2022.
 - [34] Scott Rixner, William J Dally, Ujval J Kapasi, et al. Memory access scheduling. In *ISCA*, 2000.
 - [35] Debendra Das Sharma. Introduction to compute express link. *IEEE Micro*, 43(3):7–8, 2023.
 - [36] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Antonio Barbalace, Maya Gokhale, and Abhishek Shrivastava. Demystifying cxl memory with genuine cxl-ready systems and devices. *ACM Transactions on Architecture and Code Optimization*, 20(4):1–26, 2023.
 - [37] Yupeng Tang, Ping Zhou, Wenhui Zhang, Henry Hu, Qirui Yang, Hao Xiang, Tongping Liu, Jiaxin Shan, Ruoyun Huang, Cheng Zhao, et al. Exploring performance and cost optimization with asic-based cxl memory. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 818–833, 2024.
 - [38] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, pages 1–17, 2015.
 - [39] Viktor Wahlgren et al. Understanding cxl performance: A deep dive into latency characteristics. In *HPCA*, 2024.
 - [40] Wm A Wulf and Sally A McKee. Hitting the memory wall: Implications of the obvious. *ACM SIGARCH computer architecture news*, 23(1):20–24, 1995.
 - [41] Zi Yan, Daniel Lustig, David Nellans, and Abhishek Bhattacharjee. Nimble page management for tiered memory systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, pages 331–345. ACM, 2019.